

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Universidad Distrital Francisco José de Caldas

Systems Engineering

Final Report: KKBOX Music Recommendation (Kaggle, WSDM 2018)

Miguel Alejandro Chavez Porras, Henry Jhonmarcos Osorio Sánchez,

Juan Andrés Jimenez Palomino, Daniel David Cuellar Cangrejo

December 12, 2025

Abstract

The present report aims to describe and analyze the design and development process of a predictive artificial intelligence (AI) model. The main objective of this model is to determine whether a user is likely to replay a particular song within one month after listening to it for the first time. To address this problem, the model uses historical user behavior data, focusing on interactions with similar tracks. The methodology involves data preprocessing, system analysis and design, and the implementation of feature engineering. This study contributes to understanding user engagement in music streaming platforms and how AI can enhance personalized recommendation systems. The results and conclusions of this research are still in progress.

Keywords: Music, System Analysis, Artificial intelligence model, user recommendation

Acknowledgements

We thank Kaggle for hosting the WSDM KKBOX challenge and providing the dataset, and the open-source community for maintaining the tools used in this analysis, including Scikit-learn, Pandas, and Flask.

Contents

List of Figures iv

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Objectives . . . 1

2 Scope 3
- 2.1 Assumptions . . . 3
- 2.2 Limitations . . . 4

3 Methodology 5
- 3.1 Simulation Implementation . . . 5

4 Results 8

5 Discussion and Analysis 10

6 Conclusions 11

References 12

List of Figures

3.1 System Architecture Overview 6
3.2 Data Ingestion 7

4.1 Results from Kaggle 9

List of Abbreviations

AUC	Area Under the Receiver Operating Characteristic Curve.
Cold-Start Problem	Difficulty in making predictions for unseen users or items.
Concept Drift	Change in the statistical properties of target variables over time.
ETL	Extract, Transform, Load process for data management.
SHAP	SHapley Additive exPlanations, used to interpret feature importance.

Chapter 1

Introduction

The evolution of music consumption has been deeply influenced by the rise of streaming platforms and the development of personalized recommendation systems. As algorithmic models have replaced traditional curators, users now enjoy unprecedented musical diversity and autonomy in their listening choices. Yet, despite these advances, major challenges remain. Recommender systems still struggle to accurately capture individual taste variability and to make reliable predictions when user data is limited or entirely new.

In response to these issues, recent research initiatives have encouraged the creation of models that better understand user behavior and engagement. This work contributes to that effort by designing an AI model that predicts whether a listener will replay a song within a defined period, using historical interaction patterns to enhance music personalization and recommendation accuracy.

Some previous answers tell us about the similarities between this music recommendation problem and the click value rate and that's because the way those two work and behaves are similar but differing the context, knowing that fact, they built the model and later the solution proposal.

1.1 Background

The KKBOX Music Recommendation System aims to predict whether a user will replay a song within one month of the first listen. The system operates on user-song interactions, including demographic data, song metadata, and contextual features (e.g., source type, listening tab). Due to its temporal split and noisy features, the dataset exhibits nonlinear and unstable patterns.

The present project analyzes the system from a systems engineering perspective, identifying its elements (users, songs, events, time, context) and proposing an architecture to manage complexity, sensitivity, and chaos through robust data validation, feedback monitoring, and retraining mechanisms.

1.2 Objectives

- Predict the probability that a user will listen again to a given song within one month.
- Identify and mitigate the sources of instability and chaos affecting model predictions
- Design a modular and self-regulating system architecture capable of handling noise, concept drift, and feedback effects.

- Evaluate performance using the AUC metric and ensure system interpretability through feature importance analysis.

Chapter 2

Scope

The study focuses on the KKBOX dataset and does not extend to other streaming platforms. Only the features provided by the competition datasets (train, test, songs, members, and song_extra_info) are considered. The scope includes data preprocessing, modeling, evaluation, and monitoring, excluding aspects such as UI design or large-scale deployment. Temporal prediction is limited to a one-month horizon.

The scope of this study is focused on predicting whether a user will replay a song within a 30-day period after the first listening event, based on historical interaction data. The research is limited to the dataset provided, which includes user listening histories, song metadata, and contextual interaction information. The system is designed to analyze patterns in user behavior and song characteristics, but it does not account for external factors such as marketing campaigns, social influences, or sudden changes in global trends. While the modular architecture allows for continuous monitoring and retraining, the scope of evaluation is constrained to conceptual tests and preliminary validation metrics, rather than full deployment in a live music streaming environment. Additionally, the study focuses on generalizable patterns rather than individual-level predictions, aiming to provide insights applicable to broader user populations rather than personalized recommendations for single users.

2.1 Assumptions

During the design and evaluation of the proposed system, several assumptions were made to ensure meaningful results. It is assumed that the historical listening data reflects typical user behavior and that past interactions can reliably indicate future song repetitions, with extreme outliers considered negligible. The ETL processes are assumed to successfully handle missing, inconsistent, or corrupted data, providing a clean and consistent dataset for model training and evaluation. Additionally, the selected features, such as user activity counts, song popularity, and temporal listening patterns, are assumed to capture the most relevant factors influencing song repetition, with irrelevant or redundant features having minimal impact. User preferences and song popularity are assumed to remain relatively stable within the 30-day prediction window, and patterns learned from the dataset are expected to generalize to similar users and songs not explicitly present in the training data. Finally, each user-song interaction is treated as an independent event, except as captured by aggregated features, allowing the system to analyze discrete interactions effectively.

- The data could need some clarification, modification, and an analysis to develop a structure of the system and later implement the needed model.
- Missing and noisy data can be effectively managed through preprocessing and filtering.

- The relations between the project and the click value rate point of view (solution from the winner of the competition) are very interesting but more importantly the approach is almost the same.
- User feedback loops are observable and measurable in model drift over time.
- The selected model (if it's a decision tree or a random forest) can approximate nonlinear interactions among features.
- Only assume songs / playlists, not concerts or new releases; those are not modeled directly.
- This system might have a very high chaotic and sensitivity response, because new users make randomness almost the truth.

2.2 Limitations

Several limitations were identified in the design and implementation of the system. The model's performance is highly dependent on the quality of input data, making robust extraction, transformation, and loading processes critical to handle missing values, inconsistencies, and noise. Predictive models are inherently sensitive to feature selection and preprocessing, meaning that small variations in encoding, normalization, or aggregation can significantly affect outcomes, which emphasizes the importance of careful feature engineering and validation. Processing large-scale datasets and training predictive models also require considerable computational resources, with increases in dataset size leading to longer training times and higher memory demands. Moreover, user listening behavior is dynamic and unpredictable, influenced by trends, new releases, and social factors, which can introduce instability in predictions and complicate cold-start scenarios. Lastly, the temporal dimension adds complexity, as changes in user preferences over time can reduce the relevance of predictions if not properly accounted for in the modeling process.
Also, we have:

- Data noise: The quantity of ISRC codes (the ones who identify a song) and unreal age values could reduce the model accuracy.
- Unpredictability: The randomness of the user can trick data that if we do not know how to handle, we might make bad predictions or not solve the problem at all.
- The new data needs to be processed to avoid any kind of duplication, or any data integrity problem
- New users and songs lack historical data, limiting prediction stability.
- Recommendations influence future listening; this could introduce a bias into the popularity of songs.

Chapter 3

Methodology

This study implements two complementary simulation modes to exercise a prototype music-replay prediction architecture: a data-driven machine learning (ML) workflow and an event-drive cellular automata (CA) prototype. Both simulations reuse the KKBox competition dataset as the base source for structural help while using little slices to preserve computational reproducibility. (that every person who tries to repeat it, finds the same outcome as the main one).

Data preparation

To develop a good simulation, first we need to clean and filter the given data, some of this data includes a user profile, song metadata, extended music information. This is taken from members.csv, songs.csv, song_extra_info.csv, train.csv (KKBox competition files). These files were merged and harmonized into a single cleaned artifact clean_kkbox.csv

Cleaning & sanitization script

membersclean.py; domain-driven rules were applied to avoid unrealistic training inputs.
Procedures included:

- Invalid bd (age) values (≤ 0 or > 100) replaced with NaN.
- Missing categorical entries (e.g., gender, genre ids, composer, lyricist, artist name) imputed as "unknown" to preserve encoding feasibility.
- Temporal fields (registration_init_time, expiration_date) converted to datetime objects.
- Duplicate user–song interactions removed to prevent circular learning loops.
- Rows with fewer than 50% valid fields filtered out (sparsity threshold).

To validate inference modules and speed iteration, a 50,000-row subset (same 22 columns) was saved and used for modular testing (accuracy.py).

3.1 Simulation Implementation

Event-driven Cellular Automata Experiment

The main objective is to Simulate local propagation dynamics of replay tendency using discrete event rules rather than learned parameters. we used a 50×50 grid (2,500 cells) initialized all

zeros (inactive) to favor event-driven dynamics. The neighborhood (8-adjacency) and the boundaries, no wrap-around (non-periodic boundaries). The event generation per iteration is a stochastic number of localized events (1–8) chosen uniformly. Each event is sampled according to the Event–Intention distribution (see Table below): Neighbor influence requires

Triggered Event	Intention
Song Play (0.14)	observation event, no direct state change
Replay Activation (0.10)	local activation to state 1
Neighbor Influence (0.35)	conditional activation if neighbors $\geq$ 3
Random Noise (0.01)	state flip perturbation

Table 3.1: Event-Intention Table

$\geq$ 3 active neighbors to trigger adoption. This threshold was selected to provide a moderate resistance to single-event activation and approximate multi-user pressure.

Data-Driven ML

In the Case of the ML, we use a classic ML prediction workflow was implemented using a Random Forest classifier to simulate replay learning and song-replay probability estimation
In the training process, we used the previously clean data, and eight core predictors were selected and encoded consistently across train/test feature space. Also the data were divided into an 80/20 stratified train–test split, preserving target proportions. **Implementation and Reproducibility** Implementation artifacts: membersclean.py, accuracy.py, model training scripts, CA simulation module (named cellular_sim.py in the prototype repository) and the ML simulation. All scripts accept a seed parameter and a configuration.

Environment: Typical Python ML stack (pandas, scikit-learn, joblib for serialization). Exact environment and versions are recorded in a requirements.txt alongside the repository for reproducibility.

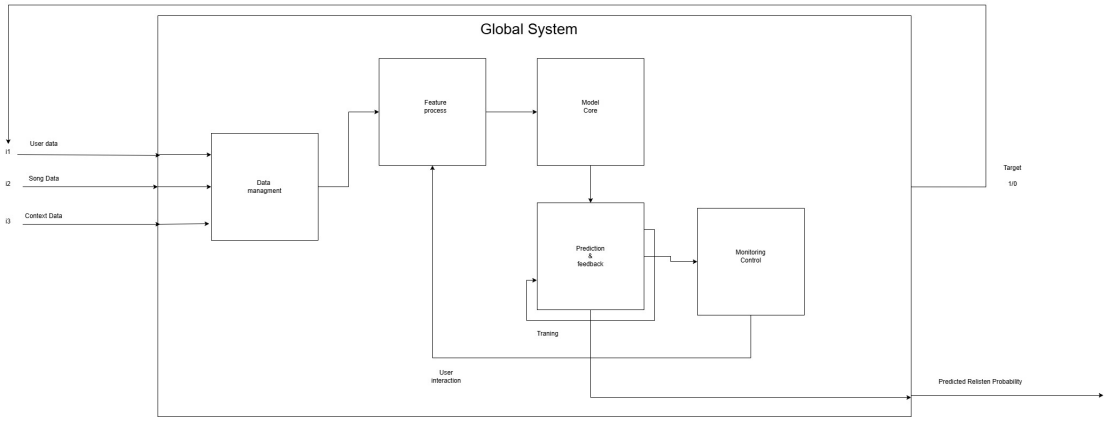


Figure 3.1: System Architecture Overview

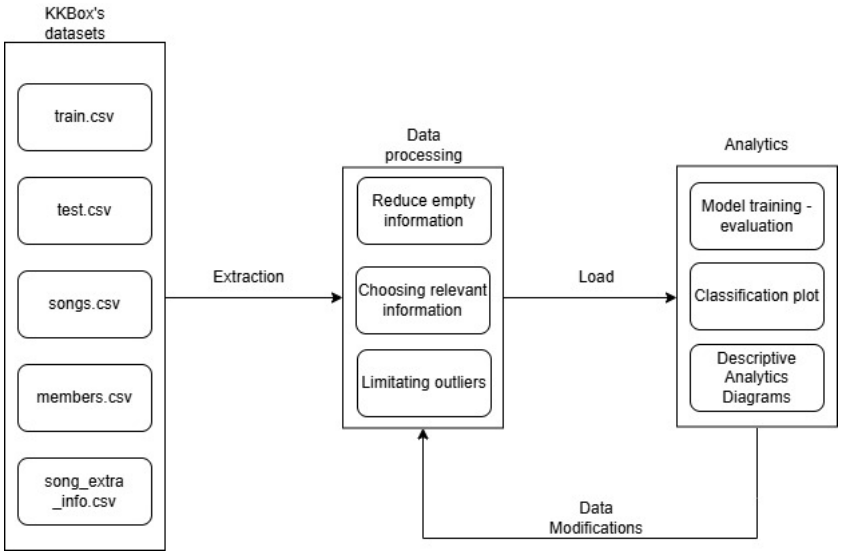


Figure 3.2: Data Ingestion

Chapter 4

Results

Using the Cellular Automata Simulation, we could say about:

The CA exhibits strong sensitivity to small probability changes: modest increases in Neighbor Influence or Replay Activation produce disproportionately large activation clusters, while increases in Random Noise can invert large active patches into inactive states.

Across repeated runs with minor probability perturbations, global activation patterns inverted or shifted dramatically. The system thus behaves like a stochastic discrete dynamical system near a transition threshold rather than a stable diffuser of activity.

Introducing damping (tested hypothetically) reduced volatility and produced more persistent activation regions; however, the baseline implementation (no decay, binary states) remained perturbation sensitive.

Using the Machine Learning Simulation, we see a stable and predictable behavior, strongly dependent on data quality, and the level of preprocessing applied. But Performance only improved once the datasets were cleaned; irrelevant noise was mitigated, and consistent category labels were enforced, enabling the model to learn meaningful patterns. Also the randomness from the model was controlled using a fixed seed for reproducible learning behavior.

$ROC - AUC = 0.6634, Accuracy \approx 0.63$

Reduced 50,000-row inference validation slice:

$ROC - AUC = 0.7345, Accuracy = 0.7435$

These results confirm that the data-driven ML predictor successfully exercised and validated ingestion $\rightarrow$ encoding $\rightarrow$ learning $\rightarrow$ probabilistic replay inference, while demonstrating the pipeline's sensitivity to data quality and categorical consistency.

The performance trend supports the design hypothesis: feature relevance and data integrity directly drives learning stability, while raw unfiltered inputs amplify chaos and degrade approximation quality.

Kaggle Result

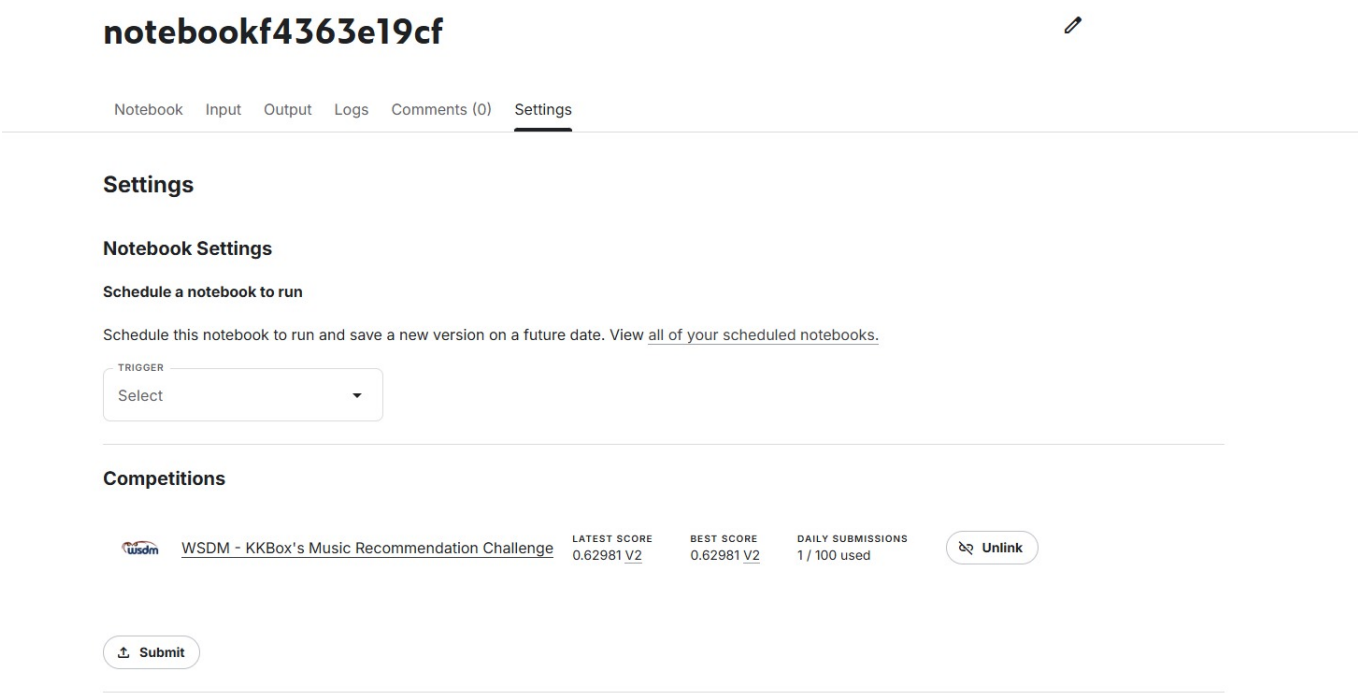


Figure 4.1: Results from Kaggle

Here we can see the effectiveness given from the Kaggle competition after we upload the proposed model, making an score of 0.63 approximate

Chapter 5

Discussion and Analysis

The results indicate that the data-driven model is sensitive to noise and uncleaned categories.

In contrast, the Cellular Automata model showed high sensitivity to small fluctuations, exhibiting chaotic-like behavior. Minor changes in event probabilities produced drastically different outcomes. Unlike the ML predictor, the CA does not learn from real data; instead, it simulates local propagation dynamics of replay tendencies based on stochastic rules.

Chapter 6

Conclusions

Dependence on Initial Conditions, the ML model depends on preprocessing data and trust the information cleaned; CA depends on event probabilities so if you change the probabilities or the clean data, the model might go crazy.
The reduced 50k slice is **useful for modular validation but may not be representative of the full data distribution**, which explains the performance of divergence between full and reduced evaluations.
Thorough cleaning, safe imputation, and controlled encoding **substantially improve model performance and reproducibility**. The pipeline reliably produced probabilistic replay predictions with moderate ROC–AUC on the full dataset and improved metrics on a reduced, cleaner slice.
Sensitivity to Noise, like ML performance decreases with noisy categories; CA drastically shifts global behavior with small perturbations.
Both models require a **millimetric calibration** to produce realistic outcomes, making the probabilities and the given and cleared data "realistic".

References

1. Celma, O. (2010). Music Recommendation and Discovery. Springer.
2. Schäfer, H., & Jannach, D. (2013). Combining Recommendations and User Decision Modeling in Music. ACM TiiS.
3. Schedl, M. (2018). Deep Learning in Music Recommendation Systems. ACM Computing Surveys.
4. Van den Oord, A., Dieleman, S., & Schrauwen, B. (2013). Deep Content-Based Music Recommendation. NIPS.
5. KKBOX WSDM Cup 2018: Kaggle Competition Description and Data Overview.